PathWeaver: Visualizing Coordination Tradeoffs in Multi-Agent Pathfinding

Team Members: Jay Parikh, Nayonika Sen, Ansh Sarin

Abstract

PathWeaver is an interactive system for studying multi-agent pathfinding (MAPF) through visualization, scenario design, and algorithm comparison. The project addresses a common instructional gap: students may understand single-agent shortest-path search, yet still lack intuition for why coordination becomes difficult once several agents share the same space over time. PathWeaver allows users to configure grids, place multiple agents, run different coordination strategies, and inspect both animations and summary metrics.

This paper presents the final PathWeaver design and a dense 15×15 case study with four agents and twenty-five obstacles. In that experiment, all four algorithms produced the same makespan and total cost, but differed substantially in conflict count and runtime. Independent planning was fastest but left four conflicts unresolved. Prioritized planning and Cooperative A* reduced the conflict count to one with modest runtime overhead, while CBS eliminated all remaining conflicts at the highest planning cost. The results reinforce the project's central claim: stronger coordination improves solution validity, but usually requires additional computation.

Introduction

Multi-agent pathfinding (MAPF) studies how multiple agents move from start states to goal states in a shared environment without colliding. The problem appears in warehouse robotics, autonomous traffic coordination, game AI, and swarm systems. Unlike single-agent search, MAPF must account for the fact that one agent's path changes the feasible search space for every other agent. A path that is optimal in isolation may become invalid once two agents try to occupy the same cell at the same time or swap positions across an edge.

That coordination burden is also what makes MAPF difficult to teach well. Textbook explanations define reservations, conflicts, and constraints, but they do not always make algorithm behavior

intuitive. PathWeaver was built to make these dynamics visible. The system lets a user choose a scenario, run an algorithm, animate the resulting motion, and compare metrics such as makespan, total cost, conflicts, and runtime. Its contribution is not a new MAPF algorithm, but an interpretable environment in which established methods can be compared directly.

Background and Related Work

The main conceptual foundation for PathWeaver comes from David Silver's paper "Cooperative pathfinding," [1]. Silver argues that planning shortest paths independently is insufficient in shared environments because agents can collide even when each local path is individually optimal. To address this, he extends pathfinding into space-time: an agent does not simply occupy a spatial coordinate (x, y), but rather a time-indexed state (x, y, t). A reservation table then marks positions already claimed by previously planned agents, allowing later searches to avoid them. This idea is directly reflected in PathWeaver's reservation-based coordination modes. In particular, our Cooperative A* implementation demonstrates the practical value of treating time as part of the search state rather than as an external simulation artifact.

Another key influence is Conflict-Based Search (CBS), introduced by Sharon, Stern, Felner, and Sturtevant [2]. CBS solves MAPF by separating the problem into two levels. At the low level, it computes individual agent paths under a set of constraints. At the high level, it detects conflicts and branches by adding new constraints to one conflicting agent or the other. This architecture is appealing because it resolves only the conflicts that actually arise rather than globally entangling all searches from the start. In PathWeaver, CBS serves as the strongest coordination baseline: it is more expensive than the reservation-based methods, but it is also better at eliminating residual conflicts that greedy planners may leave unresolved.

These two references provide complementary perspectives on MAPF. Cooperative A* shows how lightweight coordination can be layered onto classical search through reservations in space-time, while CBS shows how explicit conflict reasoning can produce higher-quality solutions at additional cost. PathWeaver places both styles in the same environment so that users can compare them with simpler baselines such as independent planning and prioritized planning.

Methods

System Overview

PathWeaver is a grid-based interactive MAPF visualizer. The environment is discrete, four-connected, and obstacle-aware. Agents have unique start and goal positions, and each agent may either move one cell per timestep or wait in place. The final interface supports both preset scenarios and user-constructed grids. The preset scenarios make it easier to run controlled comparisons, while the custom grid builder makes the system flexible enough for demonstrations and experimentation.

The top-level dashboard combines a settings panel, a central grid editor and animation view, and a metrics area so that qualitative inspection and quantitative comparison remain in the same screen.

The custom grid workflow is particularly important for the educational value of the project. A user can choose grid dimensions, add or remove obstacles, enter start and goal coordinates, and then run the planner on the constructed environment. This allows the system to function both as a reproducible experimental platform and as an exploratory sandbox.

Algorithms Implemented

PathWeaver compares four planning strategies that represent progressively richer forms of coordination.

Independent planning. In the independent baseline, each agent plans without reserving space for the others. This method is attractive because it is simple and usually fast. However, it does not enforce joint validity, so collisions can remain in the final plan. In the interface, this algorithm serves as the baseline against which the benefits of stronger coordination can be measured.

Prioritized planning. Agents are planned sequentially, and each completed path is added to a reservation structure so that later agents avoid already claimed states. This method often removes obvious collisions while staying relatively lightweight, but the outcome depends on planning order.

Cooperative A*. Cooperative A* extends the reservation idea using a time-indexed state space. Rather than planning only through geometry, it plans through geometry over time. This is a stronger match for MAPF because it explicitly represents when a cell is occupied, not only where.

In PathWeaver's implementation, Cooperative A* acts like a more coordination-aware version of prioritized planning. It usually improves collision handling while remaining much cheaper than CBS. In this implementation, Cooperative A* uses vertex-only reservations and does not prevent swap conflicts, making it a deliberate comparison point for evaluating the impact of edge constraint checking.

Conflict-Based Search (CBS). CBS first finds candidate paths, detects the earliest conflict, and then branches on constraints to remove that conflict. This gives it the best chance of producing a conflict-free solution, but also makes it the slowest method in the comparison. Because PathWeaver is intended to remain interactive, the implementation is treated as a practical baseline rather than a fully optimized industrial solver.

Table 1 summarizes the qualitative roles of the four methods as they appear in PathWeaver.

Table 1: Algorithm roles in PathWeaver

Algorithm	**Coordination Style**	**Expected Strength / Weakness**
Independent	None beyond individual shortest-path search	Fast and simple, but may leave vertex or edge conflicts unresolved
Prioritized	Sequential planning with reservations	Often removes obvious conflicts cheaply, but depends on ordering
Cooperative A*	Reservation-based space-time planning	Stronger timing awareness than simple prioritization, but still greedy
CBS	Explicit conflict detection and constraint branching	Best conflict resolution quality in this project, but highest runtime cost

Experimental Design

The detailed quantitative case study in this paper is drawn from the dense scenario visible in the completed dashboard screenshots. This scenario uses a 15×15 grid with four agents and twenty-five obstacles. It is a useful stress case because the obstacle pattern limits maneuvering freedom while still leaving enough open space for meaningful route competition.

The metrics reported in the interface are:

Makespan. The length of the longest individual path. This measures how long the group takes to finish.

Total cost. The sum of all path lengths. This measures overall travel effort across all agents.

Conflicts. The number of unresolved collisions observed in the generated plan. In PathWeaver, these include both vertex conflicts and edge-swap conflicts.

Runtime. The wall-clock planning time in milliseconds reported by the interface for the selected algorithm.

Per-agent breakdown. For at least some views, the dashboard also reports per-agent path length and wait behavior, which helps explain how the aggregate metrics were produced.

These metrics were chosen because they represent different evaluation priorities. Makespan reflects group completion time, total cost reflects cumulative travel efficiency, conflicts reflect solution validity, and runtime reflects computational feasibility. Looking at only one metric would produce an incomplete comparison. For example, two algorithms may have the same makespan and cost but differ sharply in conflict count, or one algorithm may eliminate conflicts while being so slow that it becomes impractical for interactive use.

Results

Dense-Scenario Case Study

Figure 2 shows the dense scenario in the final PathWeaver interface. It captures a single-algorithm view for Cooperative A*, including the metric panel and per-agent table. Additional screenshots from the same dashboard were used to extract the comparison results discussed below.

The first notable result is that all four algorithms achieved the same makespan and total cost in this experiment. Each produced a makespan of 28 and a total cost of 88. This means that the visible differences between the algorithms did not come from shorter or longer routes in the aggregate. Instead, the differences came from how safely and how expensively those routes were produced.

The second notable result is the conflict pattern. Independent planning left four conflicts unre-

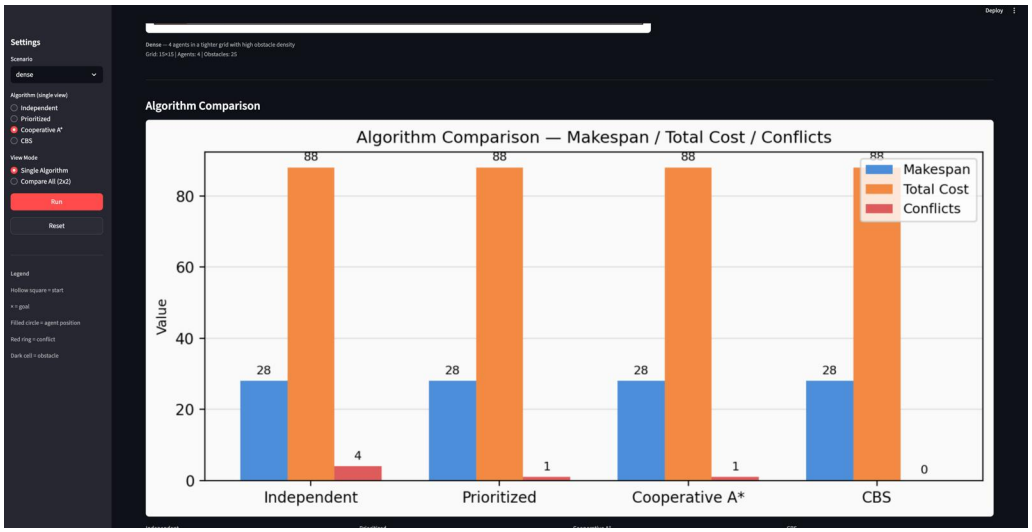

Figure 1: Single-algorithm view for Cooperative A*. The dashboard reports makespan, total cost, conflict count, runtime, and a per-agent path-length breakdown.

solved, which is the clearest demonstration in the project that individually good paths do not automatically compose into a valid multi-agent plan. Prioritized planning reduced the conflict count from four to one. Cooperative A* achieved the same conflict count of one, suggesting that the reservation-based methods captured most of the important interaction structure in this scenario. CBS then removed the final remaining conflict, producing a fully conflict-free plan.

The third important result concerns runtime. Even though all algorithms achieved the same makespan and cost, they did not require the same computational effort. Independent planning completed in 1.5 ms, Cooperative A* in 2.9 ms, prioritized planning in 4.4 ms, and CBS in 27.4 ms. The strongest conflict-handling method was therefore also the most expensive in planning time.

Table 2 converts the screenshot metrics into an explicit comparison table.

Table 2: Dense-scenario comparison from the final dashboard (15×15 grid, 4 agents, 25 obstacles)

Algorithm	**Makespan**	**Total Cost**	**Conflicts**	**Runtime (ms)**
Independent	28	88	4	1.5
Prioritized	28	88	1	4.4
Cooperative A*	28	88	1	2.9
CBS	28	88	0	27.4

This table reveals an important interpretive point. Because makespan and total cost are identical across all four methods, a naive summary might incorrectly conclude that the algorithms are equivalent. They are not. The independent baseline is clearly worse if conflict-free coordination is a requirement, and CBS is clearly stronger if perfect validity matters. The reservation-based methods occupy the middle ground: they preserve the same global efficiency as the baseline while removing most, but not all, conflicts at modest additional cost. In other words, PathWeaver shows that route-length metrics alone do not fully characterize MAPF quality.

The Cooperative A* metrics panel also exposes useful per-agent structure. In the captured run, the four agents had path lengths 28, 28, 16, and 16. This suggests an asymmetric burden distribution in the scenario: two agents travelled substantially farther than the other two. Such per-agent information is useful because equal total cost can still hide unequal individual experiences.

Graph-Based Comparison

The final interface also includes a chart view that plots makespan, total cost, and conflicts for all algorithms in the same frame. Figure 3 shows that comparison. This graph is especially effective because it compresses the full experiment into one visual summary. All four algorithms align exactly on makespan and total cost, while the conflict bars differ sharply. The graph therefore communicates, almost immediately, that the major distinction in this scenario is not path length but coordination quality.

From an interface-design perspective, this chart is one of the strongest parts of the final system. A user does not need to inspect several animations to understand the broad outcome. The chart summarizes the result at the experiment level, while the detailed views explain how that result arose.

Conclusions

Interpretation and Discussion

The dense-scenario results support several broader conclusions about coordination in MAPF.

First, independent planning is a useful baseline precisely because it fails in meaningful ways. It

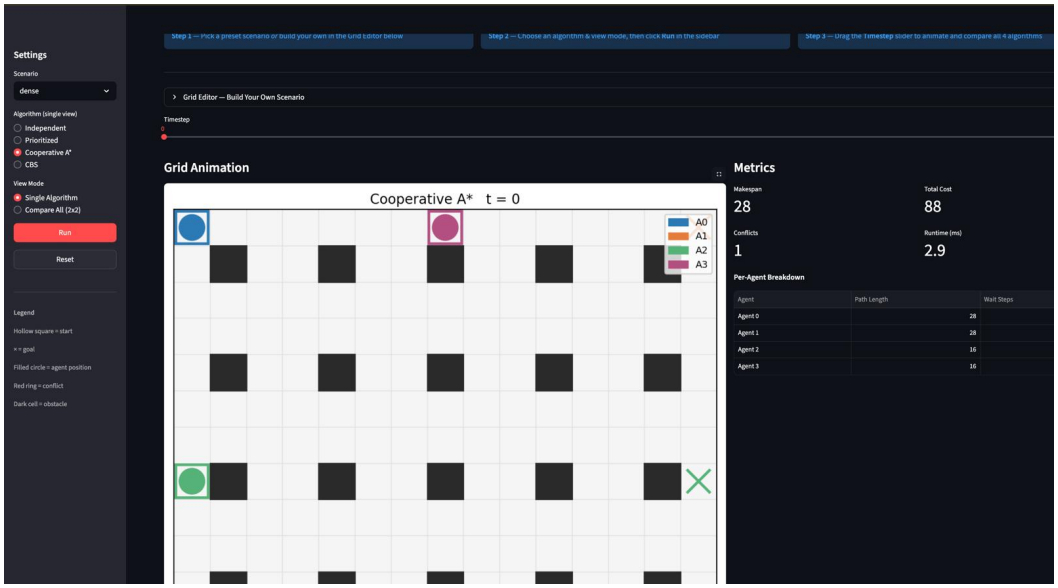

Figure 2: Algorithm-comparison chart from the final dashboard. In the dense scenario, the most meaningful separation between algorithms is conflict count rather than makespan or total cost.

shows that shortest paths for individual agents are not enough when paths overlap in time and space. In the dense experiment, the independent planner found routes with the same aggregate cost as every other method, but those routes left four conflicts unresolved. The lesson is straightforward: route efficiency does not guarantee joint feasibility.

Second, reservation-based methods can capture much of the value of coordination at relatively low cost. Both prioritized planning and Cooperative A* reduced the conflict count to one while keeping runtime in the low single-digit millisecond range. In interactive settings, that is a compelling tradeoff. A user gets dramatically improved coordination without paying the large runtime penalty associated with full conflict-based branching.

Third, CBS demonstrates why explicit conflict reasoning matters. The final remaining conflict that survived the reservation-based methods was removed by CBS, and only by CBS, in the captured experiment. This reinforces the theoretical appeal of the method: when greedy commitments are insufficient, conflict branching can recover solution validity. At the same time, the runtime increase from 2.9–4.4 ms to 27.4 ms shows why CBS is not always the default practical choice.

Fourth, the project highlights the importance of metric selection. If the interface had shown only makespan and total cost, the user would have missed the most important algorithmic difference in

the scenario. Because PathWeaver also reports conflicts and runtime, it makes visible the deeper tradeoff between efficiency, safety, and computational effort. This is one of the strongest arguments for the project as a final deliverable: it is not just an implementation of search, but a better lens for understanding search.

Limitations

Although the final system is effective as a comparative tool, it also exposes several limitations.

Scenario coverage. The detailed evaluation in this paper is based on one final dense-dashboard experiment. A stronger empirical study would include a broader collection of logged runs across multiple map sizes, obstacle densities, and agent counts.

Runtime scalability. The screenshots already suggest that CBS becomes much slower than the other methods on nontrivial cases. This is expected, but it means the current system should be understood as an educational and experimental platform rather than a production-scale planner.

Metric depth. The current interface reports useful summary metrics, but there is room to go further. Wait steps, detour ratios, fairness measures, or conflict timelines would make the analysis even richer, especially for multi-run benchmarking.

Implementation scope. PathWeaver compares four representative coordination methods, but the MAPF literature is broader than that. Suboptimal bounded-search methods, windowed planning, or learning-guided approaches could produce interesting additional comparisons.

Future Work

Future work should focus on three directions: automated benchmarking across many runs, richer metrics such as waiting time and fairness, and algorithmic refinement. CBS could be improved with better conflict ordering or caching, Cooperative A* could incorporate stronger edge-aware reservations, and prioritized planning could experiment with dynamic agent ordering.

Conclusion

PathWeaver achieves its central objective: it makes multi-agent pathfinding understandable through visualization, interaction, and metrics. In the dense 15×15 case study, all four algorithms achieved the same makespan and total cost, but differed strongly in conflict count and runtime. That result captures the core message of the project: MAPF quality cannot be understood by path length alone; it must also be understood in terms of validity and computational cost.

For that reason, the strongest contribution of PathWeaver is interpretability. Independent planning, prioritized planning, Cooperative A*, and CBS are not just implemented; they are made legible. As a final AI project, PathWeaver succeeds both technically and pedagogically by turning algorithmic tradeoffs into something users can directly see and compare.

Works Cited

References

[1] D. Silver, "Cooperative pathfinding," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, vol. 1, pp. 117–122, 2005.

[2] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.

[3] N. Sen, A. Sarin, and J. Parikh, *PathWeaver* [Computer software]. GitHub. Available: https://github.com/nayonikasen/pathweaver

[4] Python Software Foundation, *Python* [Computer software]. Available: https://www.python.org/

[5] Pygame Community, *Pygame* [Computer software]. Available: https://www.pygame.org/

[6] Streamlit Inc., *Streamlit* [Computer software]. Available: https://streamlit.io/

[7] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.